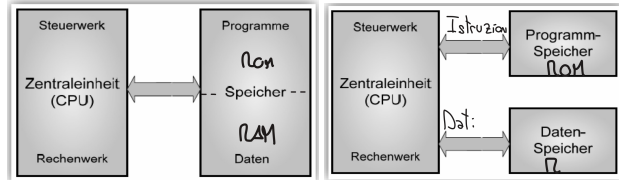


Quick Reference

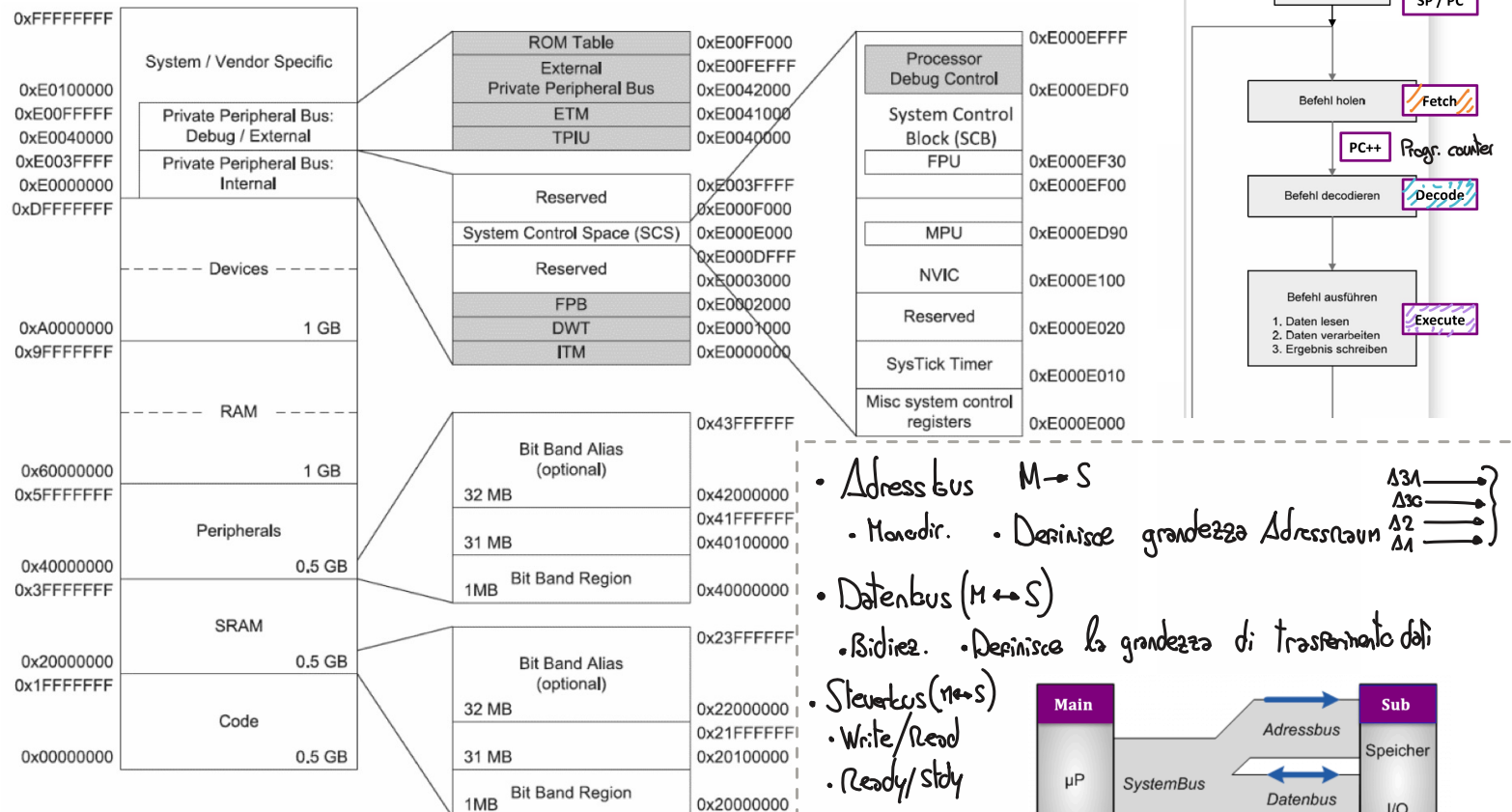
Lars Kamm, Jan Wendler



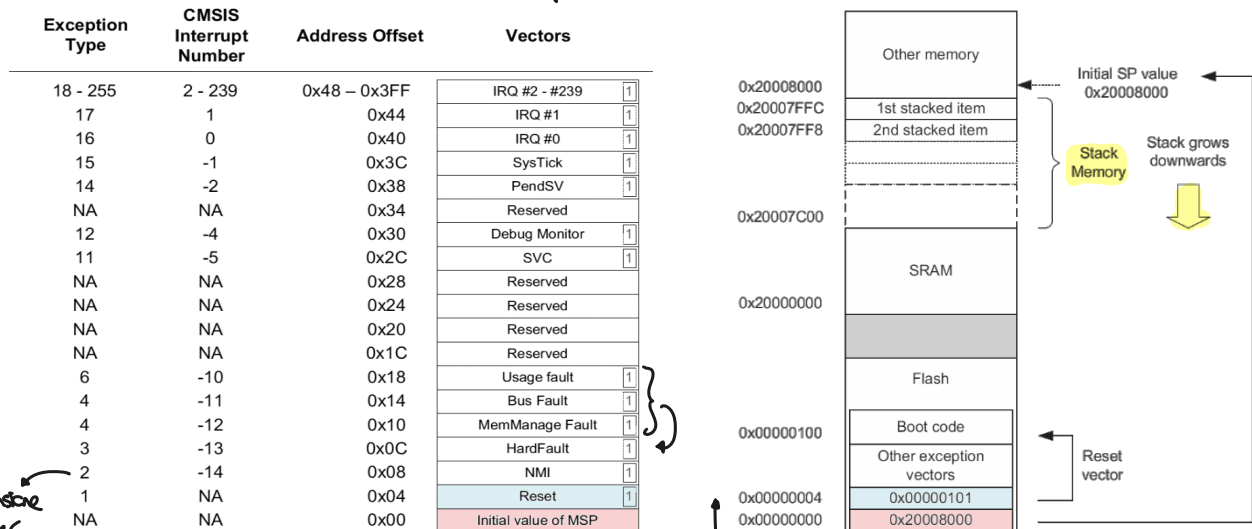
- La grandezza della ALU definisce la grandezza di un Word
- RISC: Pochi comandi,

1 ARM CORTEX-M MEMORY ORGANIZATION

1.1 ARM Cortex-M Memory-Map

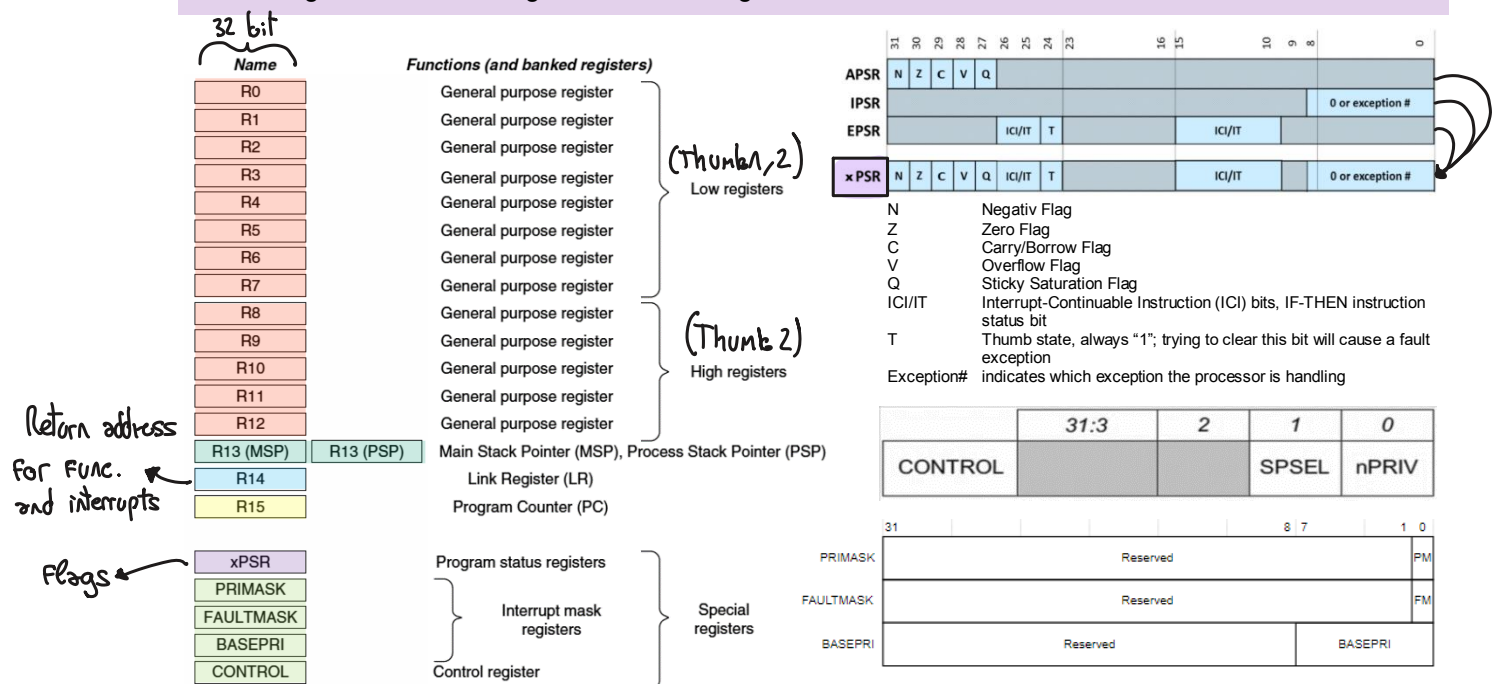


1.2 Vector-Table & Reset-Sequence (Processo di avvio)

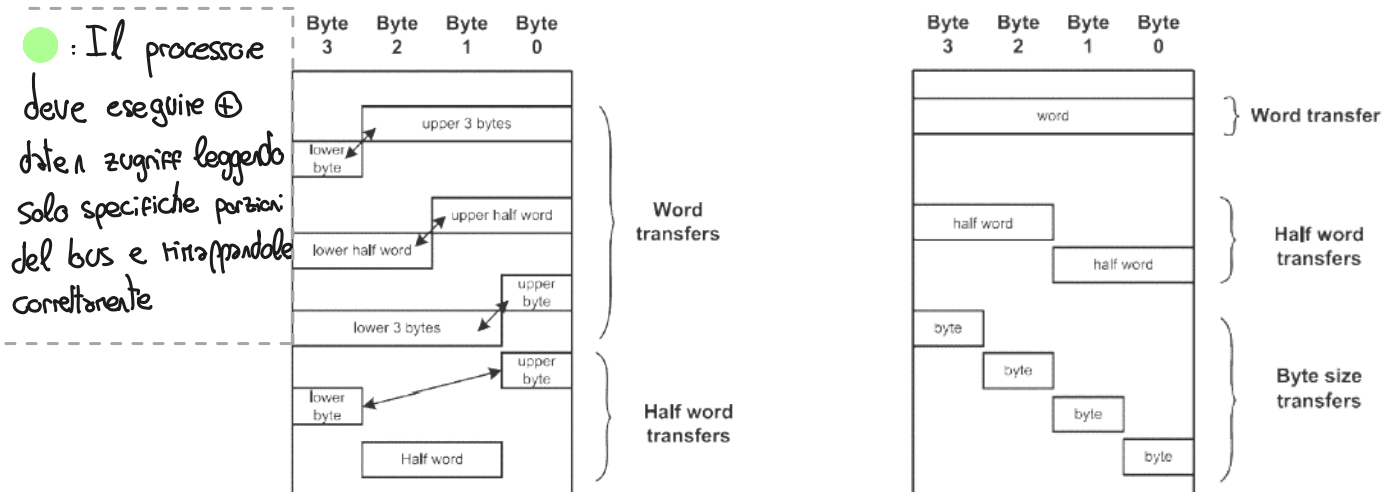


Anomalia tensione
watchdog timer

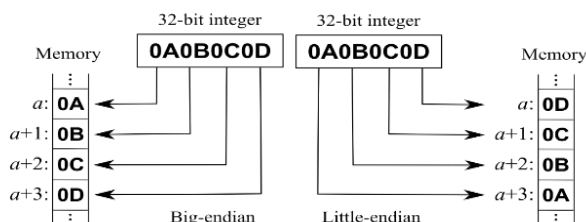
1.3 Register-Set & Program Status Register



1.4 Alignment (Cortex-M => Bus indirizzi di 32 bit (4 byte))



1.5 Endianess (Cortex-M => Little-endian)



Endianess betrifft nur Zahlenformate die grösser als ein Byte sind (Halfword, Word, Doubleword).

Little-Endian: Least Significant Byte befindet sich an der tiefsten Adresse.

Big-Endian: Most Significant Byte befindet sich an der tiefsten Adresse.

1.6 Speichergrossen

Zur Angabe der Speicherkapazität als Anzahl von Bits, Bytes oder Codewörtern verwendet man in der Informatik in Anlehnung an die Physik die Bezeichnungen Kilo (K), Mega (M), Giga (G) und Tera (T). Diese beziehen sich auf das Binärzahlensystem:

$$1K = 2^{10} = 1'024$$

$$1M = 2^{20} = 1'024K = 1'048'576$$

$$1G = 2^{30} = 1'024M = 1'048'576K = 1'073'741'824$$

$$1T = 2^{40} = 1'024G = 1'048'576M = 1'073'741'824K = 1'099'511'627'776$$

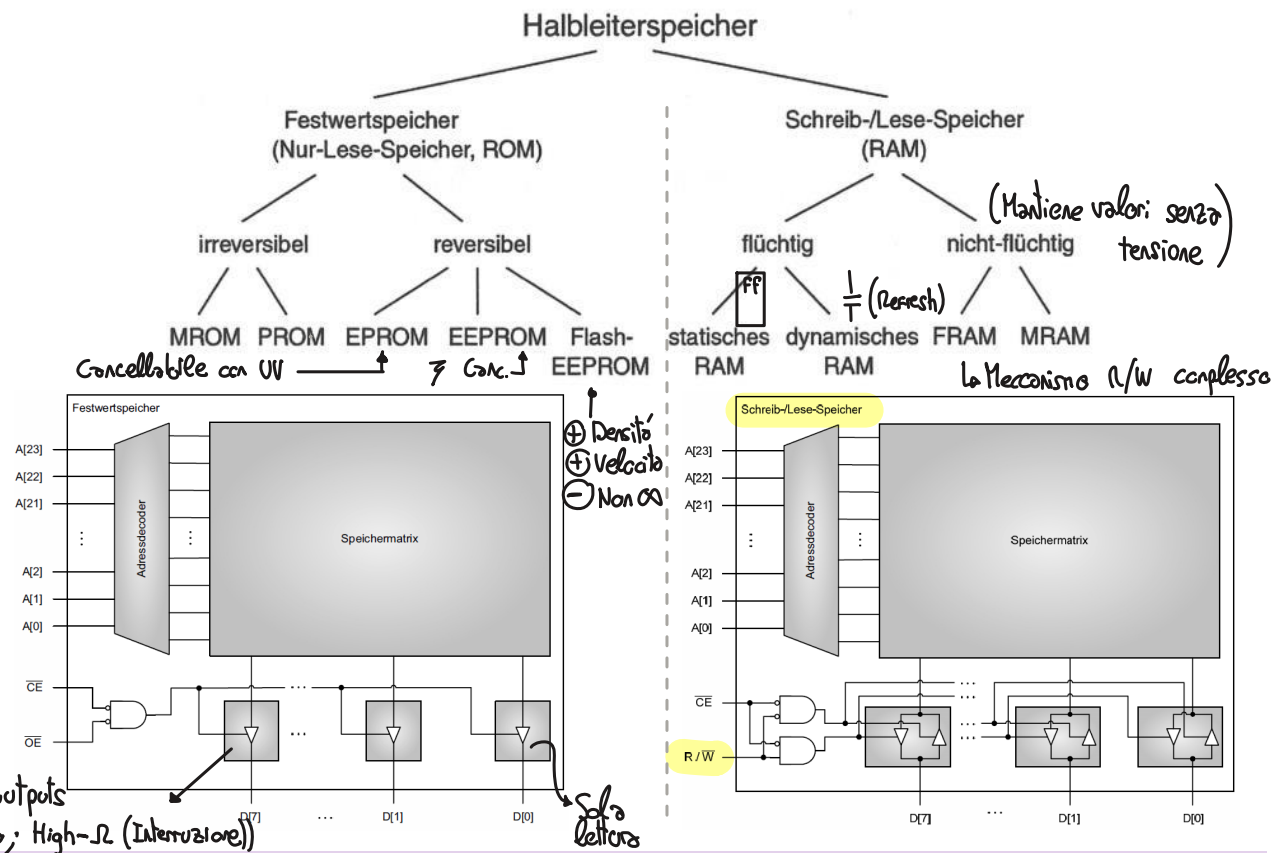
Memory mapped

↳ Gemeinsamer Adressraum für I/O und Speicher

Isolierte Adressierung

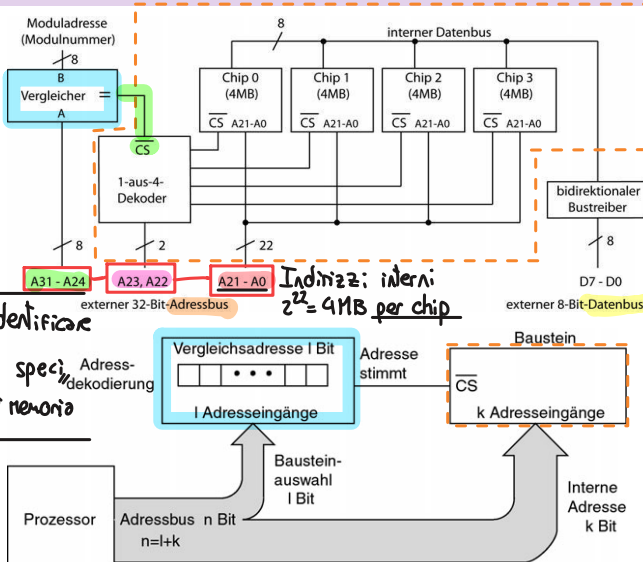
↳ Separater Adressraum für I/O und Speicher

1.7 Speicherarten

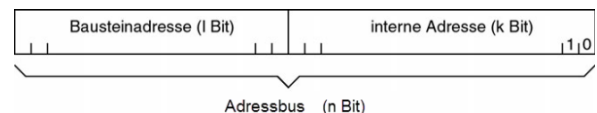


1.8 Adressdekodierung

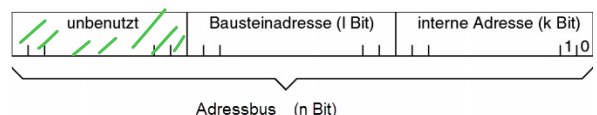
Modulo di memoria



Esempio $(22+2) + 8 = 32$
 $k + l = n$



$$k + l < n$$

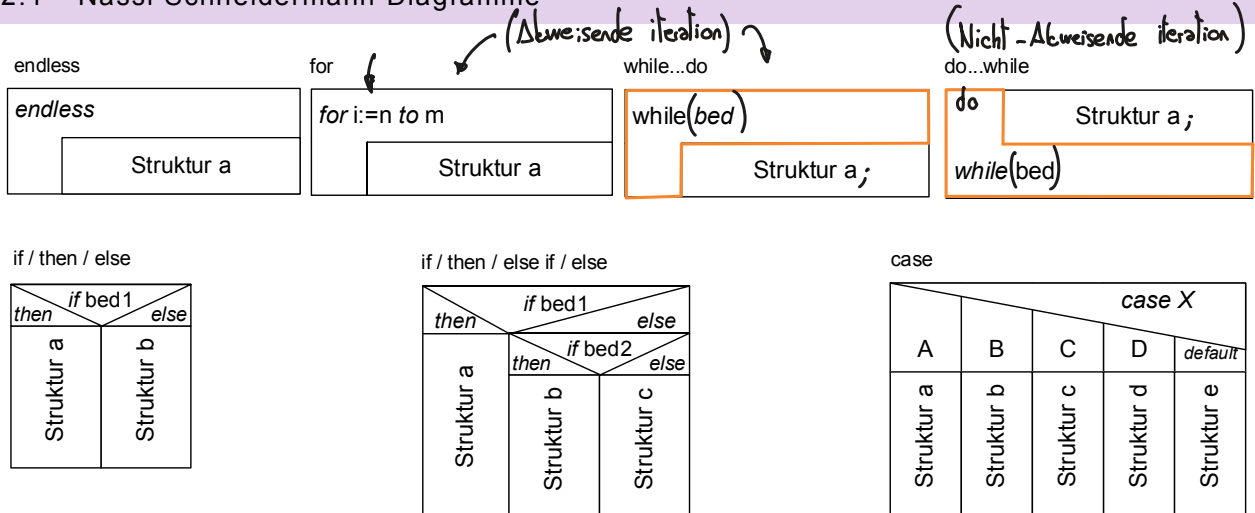


(Chip = contenitore di memoria)

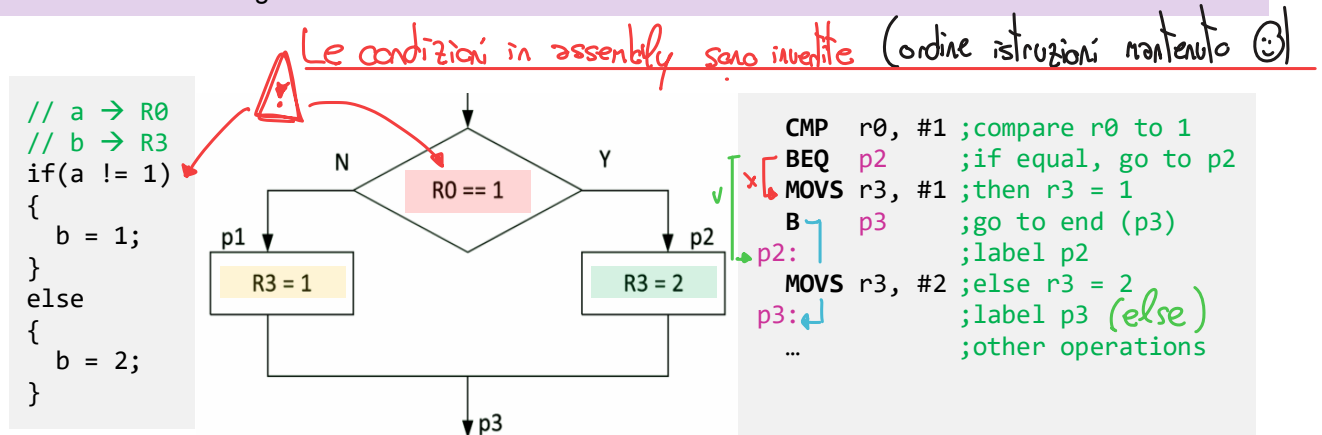
CS = Chip select (accensione chip)

2 KONTROLL- & SELEKTIONSSTRUKTUREN

2.1 Nassi Schneidermann Diagramme



2.2 Entscheidungen treffen



In Assembler-Sprache ist eine Entscheidung praktisch immer ein 2-stufiger Ablauf

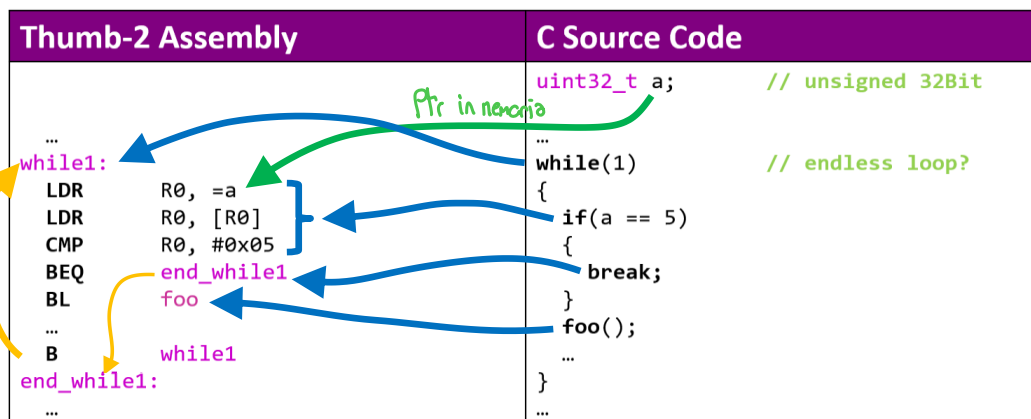
1. Benötigte Flags ermitteln

Vergleich (**Compare**) vornehmen, indem zwei Werte voneinander subtrahiert werden. Bei der Differenzbildung ist das Resultat nicht wichtig, sondern nur dessen Eigenschaften (**Flags**).

2. Zugehörige Sprünge ausführen

Im zweiten Schritt werden mit den Flags bedingte Sprünge (**B{cond}**) ausgeführt

2.3 Umsetzung von C/C++ Strukturen In questo esempio non viene eseguita l'inversione (caso speciale)



Normalmente viene invertita la cond. per "saltare" subito.
visto che ora tramite break bisogna saltare proprio se 'if'

nei loop la condizione non è invertita

3 ARM CORTEX-M0(+) INSTRUCTION-SET

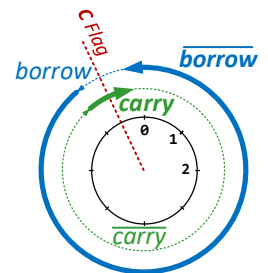
3.1 Symbole & Appendix

Symbols	Meaning
R_d, R_m, R_n	represent 32-bit registers
$\{R_d\}$	if R_d is present R_d is destination, otherwise R_n
$\{S\}$	if S is present, instruction will set condition codes (Aggiornamento flags)
$\{cc\}$	optional logical condition, condition code suffix (EQ, NE, GT, LT, etc.)
$\#imm3$	any value in the range: 0...7
$\#imm5$	any value in the range: 0...31
$\#imm7$	any value in the range: 0..127
$\#imm8$	any value in the range: 0..255
$\#imm10$	any value in the range: 0..1023
$\#imm11$	any value in the range: 0..2047
label	any address within the ROM of the microcontroller; offset range relative to PC: B label -2 KB to +2 KB B<cc> label -256 bytes to +255 bytes BL label -16 MB to +16 MB (32-bit befehl)
$\{reglist\}$	List of registers, sequence is not relevant (Ogni bit della lista indica un registro)

Parentesi grosse: opzionali

3.2 Bedingte Ausführung & Code-Zusätze {cc} (Branches)

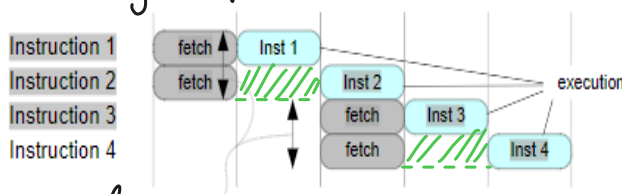
cc	Meaning	Flags (APSR) / Requirements	Binary	uint	int
EQ	Equal	Z = 1	0000	X ==	X ==
NE	Not equal	Z = 0	0001	X !=	X !=
CS or HS	Carry set, Unsigned ≥ carry OR borrow	C = 1	0010	X >=	
CC or LO	Carry clear, Unsigned < carry OR borrow	C = 0	0011	X <	
MI	Minus/negative	N = 1	0100		X -
PL	Positive or zero (non-negative)	N = 0	0101		X +
VS	Overflow	V = 1	0110		X
VC	No overflow	V = 0	0111		X
HI	Unsigned > ("Higher")	C = 1 && Z = 0	1000	X >	
LS	Unsigned ≤ ("Lower or Same")	C = 0 Z = 1	1001	X <=	
GE	Signed ≥ ("Greater than or Equal")	N = V	1010		X >=
LT	Signed < ("Less Than")	N ≠ V	1011		X <
GT	Signed > ("Greater Than")	Z = 0 && N = V	1100		X >
LE	Signed ≤ ("Less than or Equal")	Z = 1 N ≠ V	1101		X <=
AL	Always (unconditional)	any	1110		



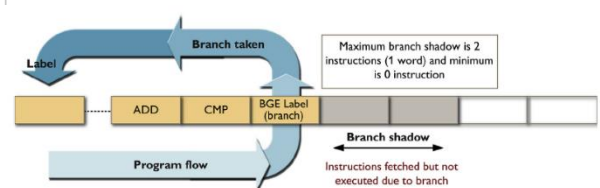
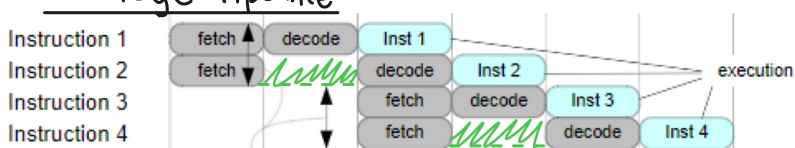
ACHTUNG:
CMP ist eine Subtraktion (Borrow)
CMN ist eine Addition (Carry)

3.3 Pipelining

2-Stage Pipeline



3-Stage Pipeline



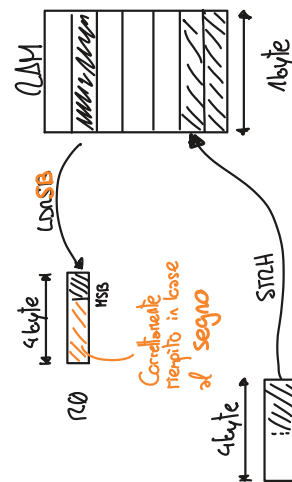
Bei Sprüngen muss die Pipeline "geleert" werden

(? Flag modificato)
logical shift left
logical shift right
Arithmetic Shift right
Rotate Shift right
(Vorz. mantendo)
Per signed val.

3.4 ARM Cortex-M0(+) Thumb Instruktionen

Group	Description	Syntax	Cycles	Flags	Instruction Code 16Bit
	Move Lo to Lo	MOV _S Rd, Rm	1	N Z C V	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Logical shift left by immediate	LSL _S Rd, Rm, #imm5	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Logical shift right by immediate	LSR _S Rd, Rm, #imm5	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Arithmetic shift right	ASR _S Rd, Rm, #imm5	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Add 3-bit immediate	ADD _S Rd, Rm, #imm3	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Add All registers Lo	ADD _S Rd, Rn, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Sub Lo and Lo	SUB _S Rd, Rn, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Sub 3-bit immediate	SUB _S Rd, Rn, #imm3	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Move 8-bit immediate	MOV _S Rd, #imm8	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Compare Immediate	CMP Rn, #imm8	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Add 8-bit immediate	ADD _S Rd, #imm8	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Sub 8-bit immediate	SUB _S Rd, #imm8	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	AND	AND _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Exclusive OR	EOR _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Logical shift left by register	LSL _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Logical shift right by register	LSR _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Arithmetic shift right by register	ASR _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Add with carry	ADCS _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Sub with carry	SBCS _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Rotate right by register	RORS _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	AND test	TST Rn, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Negate	RSBS _S Rd, Rm, #0	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Compare Lo to Lo	CMP Rn, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Compare Negative	CMN Rn, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Multiply	ORRS _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Bit clear	MULS _S Rd, Rm	1 or 32 ⁽¹⁾	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Move NOT	BICS _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Add Any to PC	MVNS _S Rd, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Add Any to Any	ADD _S PC, Rm	3	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Add address from SP and register	ADD _S Rd, Rm (AND Rd, Rd, #0)	1	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Compare Any to Any	ADD _S Rd, SP	1	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Move Any to PC	CMP Rn, Rm	1	?	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Branch With link and exchange	MOV _S Rd, Rm	2	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Load PC-relative	BX Rm	2	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Word, register offset	LDL _S Rd, <label>	2 or 1 ⁽²⁾	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Halfword, register offset	STR _S Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Byte, register offset	STRB _S Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Signed byte, register offset	LDR _S Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Word, register offset	LDRD _S Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Halfword, register offset	LDRH _S Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Byte, register offset	LDRB _S Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	Signed halfword, register offset	LDRSH _S Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Load/store



Meaning	Data type
0..4'294'967'295 or -2'147'483'648..+2'147'483'647	32-bit word
0..255	Unsigned 8-bit byte
-128..+127	Signed 8-bit byte
0..65'535	Unsigned 16-bit halfword
-32'768..32'767	Signed 16-bit halfword

Per store non importa pu il valore viene semplicemente tagliato

(1) Depends on multiplier implementation
(2) 2 if to AHB interface or SCS, 1 if to single-cycle I/O port.
(3) N is the number of elements in the list
(4) N is the number of elements in the list including PC or LR.



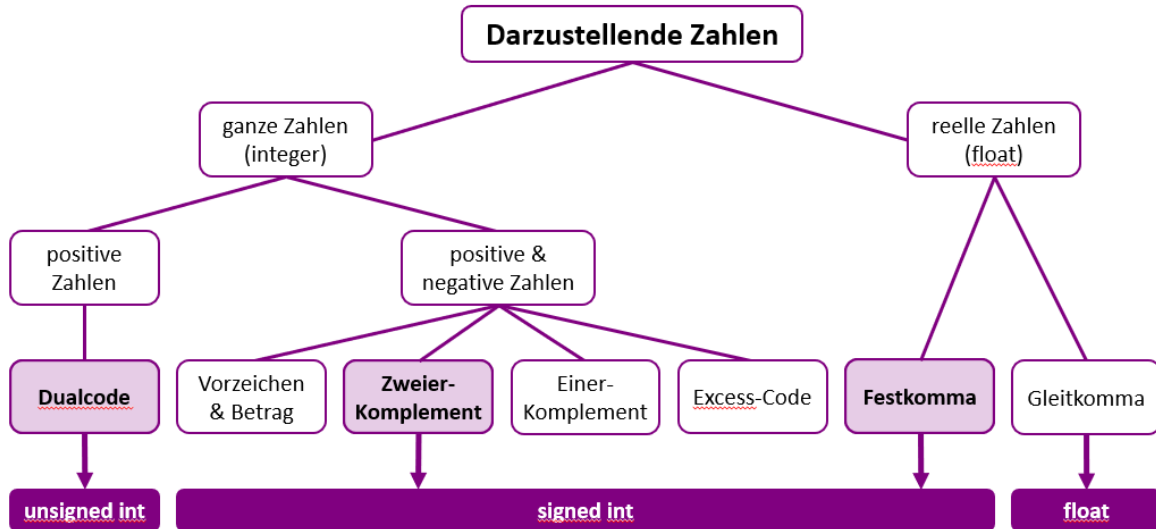
Multifache Datentransfers

- il 1° serve per aggiornare l'indirizzo R_n (dopo pointer a primo spazio libero dopo array)
- STM solo versione con !
- R_n base salvata & memoria addebi basati

- $\mathbb{R}^n \oplus \mathbb{R}^n$ is isomorphic to $\mathbb{R}^{2n}$ as vector spaces

Viene usato quando il secondo termine è negativo ($\sigma =$

4 ZAHLENSYSTEME



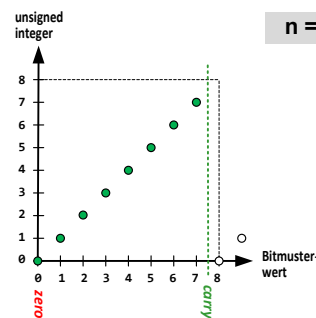
4.1 Vorzeichenlose Zahlen

Dualcode (unsigned int)

$$W = \sum_{i=0}^{n-1} d_i \cdot 2^i$$

$d_{n-1} \quad d_{n-2} \quad \dots \quad d_2 \quad d_1 \quad d_0$

Relevante Flags: Zero, Carry/Borrow



Bereich
 $0 \dots 2^n - 1$

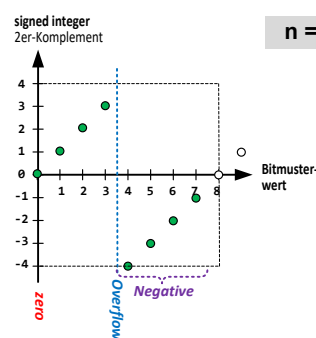
4.2 Vorzeichenbehaftete Zahlen

Zweierkomplement (signed int) → Comportamento ALU ⊕ unsigned

$$W = \left(\sum_{i=0}^{n-2} d_i \cdot 2^i \right) - d_{n-1} \cdot 2^{n-1}$$

$d_{n-1} \quad d_{n-2} \quad \dots \quad d_2 \quad d_1 \quad d_0$

Relevante Flag: Zero, Negativ, Overflow
MSB = 1



Bereich
 $-2^{n-1}, \dots, 2^{n-1} - 1$

4.3 Zweierkomplement-Negierung

Vorgehensweise:

- 1) Vorzeichenbehaftete Zahl in Binärcode darstellen
- 2) Alle Bits einzeln invertieren
- 3) Binäre Addition mit dem Wert +1
- 4) Resultat entspricht der negierten Vorzeichenbehaftete Zahl

Beispiel: $-3_{10} = 101_2$
Invertieren: 010_2
+1: 001_2

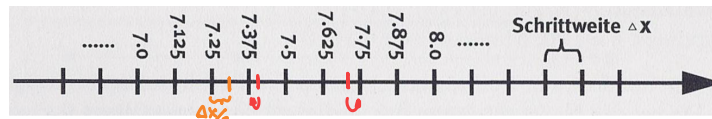
 011_2
=====

Resultat: $+3_{10}$

$$2^{-1} = 0,5 ; \quad 2^{-2} = 0,25 \quad 2^{-3} = 0,125 \quad , \quad 2^{-4} = 0,0625 ; \quad 2^{-5} = 0,03125$$

$$2^{-6} = 0,015625$$

$$d_n \cdot 2^n + d_{n-1} \cdot 2^{n-1} + \dots + d_0 \cdot 2^0$$



4.4 Festkomma-Zahlen

Bisogna correggere i numeri (Nearest) ⚠

Dec. Bin.
Da 10 a 0b calcolare 1 passo in
nella parte dopo virgola e vedere se serve
rounding

Vorzeichenlose Fixed-Point-Zahlen

Allgemeine Formel (Dualcode als Basis)

$$W = \sum_{i=-m}^{n-1} d_i \cdot 2^i$$

$d_{n-1} d_{n-2} \dots d_1 d_0, d_{-1} d_{-2} d_{-m}$

Wertebereich: $0.0 \dots 2^n - 2^{-m}$

(Auflösung)
Schrittweite: $\Delta X = 2^{-m}$

Max. Diskretisierungsfehler: $\frac{\Delta X}{2} = 2^{-(m+1)}$
(Quantisierungsfehler)

4.5 IQ-Zahlenformate

Immer signed ⚠

Integer/Quotient-Darstellung

Numero cifre decim. →
Signed Fixed-Point mit $n = I$ und $m = Q$.

$$W = -d_{I-1} \cdot 2^{I-1} + \sum_{j=0}^{I-2} d_j \cdot 2^j + \sum_{i=1}^Q d_{-i} \cdot 2^{-i}$$

$d_{I-1} d_{I-2} \dots d_1 d_0, d_{-1} d_{-2} d_{-Q}$

Werteb. $-2^{I-1} \dots 2^{I-1} - 2^{-Q}$

Vorzeichenbehaftete Fixed-Point-Zahlen

Allgemeine Formel (2er-Komplement als Basis)

$$W = \left(\sum_{i=-m}^{n-2} d_i \cdot 2^i \right) - d_{n-1} \cdot 2^{n-1}$$

$d_{n-1} d_{n-2} \dots d_1 d_0, d_{-1} d_{-2} d_{-m}$

Wertebereich: $-2^{n-1} \dots 2^{n-1} - 2^{-m}$

Schrittweite: $\Delta X = 2^{-m}$

Max. Diskretisierungsfehler: $\frac{\Delta X}{2} = 2^{-(m+1)}$

Fractionale Zahlen (oder Spezialfall IQx)

$$W = -d_0 + \sum_{i=1}^Q d_{-i} \cdot 2^{-i}$$

$d_0, d_{-1} d_{-2} \dots d_{-Q}$

Wertebereich: $-1.0 \leq W \leq +1.0 - 2^{-Q}$

Vorzeichen: Koeffizient d_0

Genauigkeit: $2^{-(Q)}$

⊕ Overflow solo quando eseguo $-1 + -1$ (Weniger Aufwand)

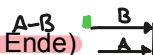
4.6 Zahlenkreise

Addition: Zahlenvektoren hintereinander reihen (Anfang auf Ende)

$A+B$



Subtraktion: Zahlenvektoren stumpf gegeneinanderstellen (Ende auf Ende)



Bsp.

$0x2 - 0x8$

Unsigned Integer

Relevante Flags: Zero, Carry/Borrow

Zahlenvektoren immer Uhrzeigersinn

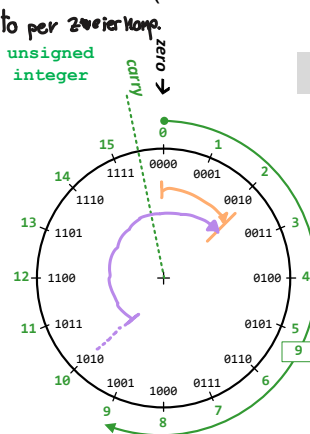
Signed Integer

(Primo bit 1) $(C_n \neq C_{n-1})$ (0...0)

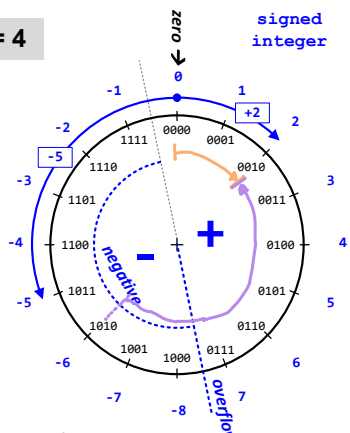
Relevante Flags: Negative, Overflow, Zero

positiv → Zahlenvektor Uhrzeigersinn

negativ → Zahlenvektor Gegenuhrzeiger



$n = 4$



■ = Corrispondente carry nella sottrazione
(Comportamento invertito rispetto a Σ)

$\begin{cases} 1 = \text{risultato} \geq 0 \\ 0 = \text{risultato} < 0 \end{cases}$ (il risultato è corretto)

Nella ruota **UNSIGNED** tutti i vettori vanno in senso orario

Nella ruota **SIGNED** i vettori ≥ 0 in senso orario, < 0 in senso antiorario (regola $\Rightarrow$ rimane)

Carry

Addizione: se $C_n = 1 \rightarrow \text{Carry} = 1$

Sottrazione: se $C_n = 0 \rightarrow \text{Carry} = 1$
(Negato)

Bsp

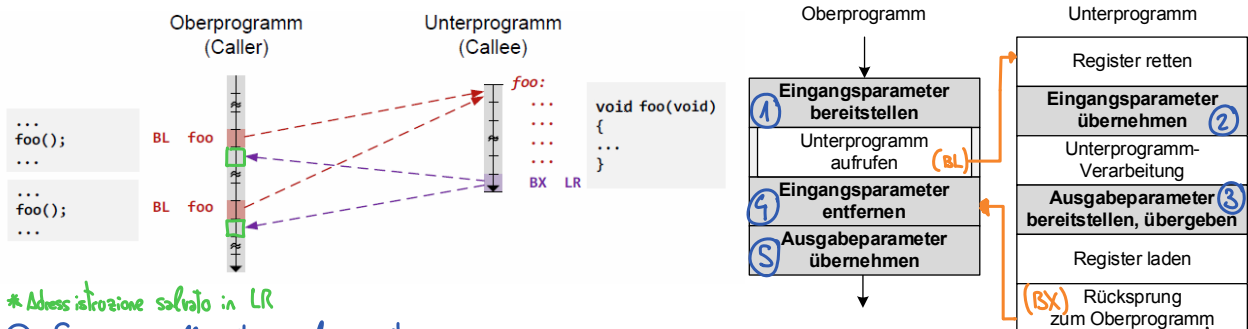
$0x2 - 0x8 \rightarrow$

$\begin{array}{r} 0010 \\ - 1000 \\ \hline 1010 \end{array}$

$\Rightarrow$
 $\begin{array}{l} Z = 0 \\ C = 0 \\ N = 1 \\ V = 1 \end{array}$

* Il compilatore aggiunge allo stack frame
LR e ulteriori registri per il calcolo, se necessario

5 SUBROUTINEN

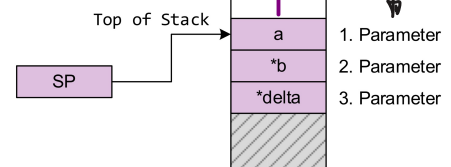


* Indirizzo salvato in LR

④ Se sono sullo stack, eliminare stack frame

* Regeln zum ARM Architecture Procedure Call Standard (AAPCS)

- Die Register **R0-R3** werden als Parameter verwendet.
- Weitere Parameter werden auf den **Stack** gelegt. (> 4 Parameter)
- Der **Rückgabewert** (≤32Bit) wird in Register **R0** übertragen.
- Ein **64Bit-Rückgabewert** wird in den Registern **R0** und **R1** übertragen.
- Funktionen müssen die Registerinhalte **R4-R11** während der Ausführung sichern und rekonstruieren.
- Die Register **R0-R3** und **R12** dürfen in der Subroutine ohne Sicherung verändert werden.
- Es wird stets ein **8-Byte Alignment** auf dem Stack eingehalten.



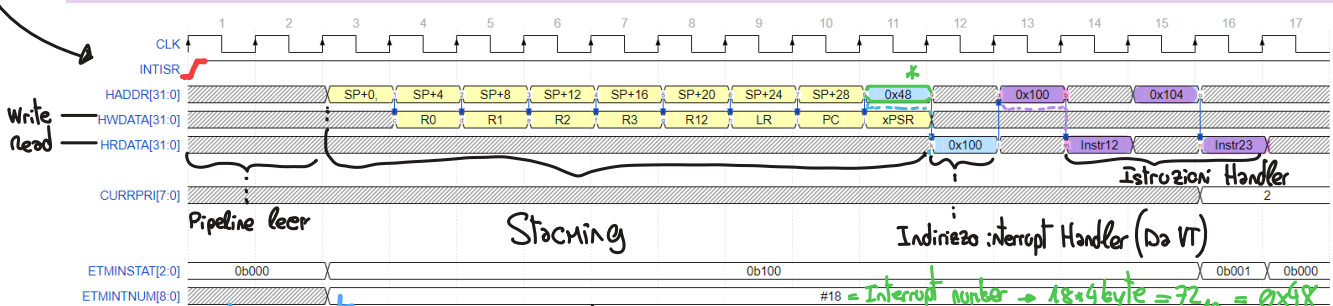
Tutti i parametri (≤32byte) vengono sempre passati come se fossero di 32 bit (int, float, double, etc.)

Register	Verwendungszweck	Scratch	Sichern
R0	Erster Funktionsparameter, Return-Wert (8-/16-/32-Bit)	Ja	
R1	Zweiter Funktionsparameter, Upper Word Return-Wert (64-Bit)	Ja	
R2, R3	Dritter und vierter Funktionsparameter	Ja	
R4-R7	Registervariablen (temporär)	Ja	Ja
R8-R11	Compilerabhängige Verwendung	Ja	Ja
R12	Funktionsinternes temporäres Speicherregister für Sprünge	Ja	
R13	Stack-Pointer (SP, Top-of-Stack)	Diese Register werden für spezielle Zwecke verwendet	
R14	Link Register (LR)		
R15	Programm Counter (PC)		

Bus HADDR é sempre 1 ciclo in anticipo rispetto a H...DATA (Address Fetching)

6 INTERRUPTS & EXCEPTIONS

6.1 Vector Fetch & Stacking ■ = Registeri salvati automaticamente



Fasi Interrupt

- Request:** (Arriva IRQ)
- Stacking & Vector-Fetch:** Register auf Stack sichern und Sprung via Vektortabelle ausführen und Active Status setzen.
- ISR Handler:** Benutzercode ausführen und am Ende den Active Status zurücksetzen
- Unstacking:** Register wieder herstellen, Stack abbauen und Rücksprung mittels LR

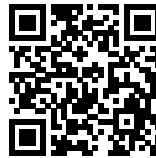
ComEng - Ergänzung

FS 2026 – Prof. Lars Kamm

Autoren:
Mirko Ratti

Version:
1.0.20260706

<https://github.com/mirkoratti/FS2026>



Inhaltsverzeichnis

1	Instruction Set Architecture (ISA)	11			
2	Logica della ALU e Program Status Register	11			
3	Componenti del Sistema (SCS)	11			
4	Stack e Gestione Memoria	11			
5	Confronto Architetture ARM Cortex	11			
5.1	Evoluzione Cortex-M0 vs M0+	11	6.5	Soluzioni alternative & Segnali IRQ	11
6	Gestione degli Interrupt (NVIC)	11	7	Ottimizzazione e Strutture Dati	11
6.1	NVIC Special Features	11	8	Adressierungsarten ARM (Cortex-M0)	12
6.2	Registri di Mascheramento	11	8.1	Register Direkt, Immediate	12
6.3	Latenza e Stacking	11	8.2	Register Indirekt (ptr)	12
6.4	Preemption vs Subpriority	11	8.3	PC-Relative & Literal Pool	12
			8.4	Registerlisten & Stack (SP-Relative)	12
			8.5	Stack Framing	12
			9	Il Processo di Compilazione	12
			9.1	Analisi del Compilatore	12
			10	Esempi	12

- 11

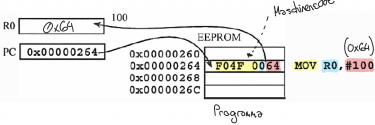
7.0.2 Efficienza delle Strutture Dati

- **Array Locali:** Molto **inefficienti**. alto numero di cicli solo per l’inizializzazione nello stack frame
- **Strutture:** È consigliabile **definire le strutture globalmente**. L’inizializzazione locale su stack comporta un overhead elevato di cicli di clock.
- **Bitfields:** Estremamente **inefficienti**. Sebbene risparmino memoria, richiedono complesse operazioni di *Read-Modify-Write* (LDR, Shift, Mask, OR, STR)

8 Adressierarten ARM (Cortex-M0)

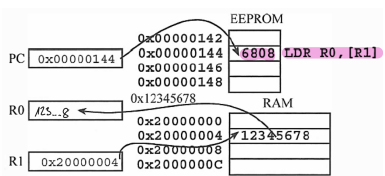
8.1 Register Direkt, Immediate

- **Immediate:** MOV R0, #100 → Valore caricato direttamente.
- **Register:** MOVS R2, R1 → Copia valore tra registri.
- **Limitazioni:** Gli immediati sono codificati nell’istruzione (es. 8-bit per MOVS).



8.2 Register Indirekt (ptr)

- **Semplice:** LDR R0, [R1] → R0 = valore all’indirizzo R1.
- **Con Offset:** LDR R0, [R1, #4] → R0 = valore all’indirizzo R1 + 4 (utile per array/struct).
- **Con Index:** LDR R0, [R1, R2] → R0 = valore all’indirizzo R1 + R2.



8.3 PC-Relative & Literal Pool

- **Problema:** Istruzioni da 32-bit non possono contenere costanti da 32-bit.
- **Soluzione:** L’assembler salva la costante nel **Literal Pool** (zona memoria vicina).
- **Accesso:** LDR R0, =0x12345678 viene tradotto in LDR R0, [PC, #offset].
- Richiedono sempre **due accessi** alla memoria

8.4 Registerlisten & Stack (SP-Relative)

- **Multiple:** LDM R7!, {R0-R3, R5} → Carica più registri partendo da indirizzo in R7.
- **Stack:** PUSH {R0-R2}, POP {R0-R2}.
- **Regola d’oro:** Il registro con numero più basso (R0) va sempre all’indirizzo di memoria più basso. L’ordine nelle graffe non conta per l’hardware.

8.5 Stack Framing

- Lo Stack Frame è lo spazio riservato nella RAM per le variabili locali di una funzione.
- **Allocazione:** SUB SP, SP, #X riserva X byte (lo stack cresce verso indirizzi bassi).
 - **Frame Pointer (R7):** MOV R7, SP per avere un riferimento fisso all’inizio del frame.
 - **Accesso:** var. vengono scritte/lette usando offset rispetto a R7, es: STR R0, [R7, #4].
 - **Deallocazione:** Prima del ritorno (BX LR), si deve pulire lo stack con ADD SP, SP, #X.

9 Il Processo di Compilazione

- Il codice sorgente viene trasformato in binario attraverso diverse fasi:
1. **Preprocessore:** Gestisce inclusioni e macro. (#define, #if, ...)
 2. **Compilatore:** Traduce il linguaggio ad alto livello in Assembly.
 3. **Assembler:** Converte Assembly in codice macchina rilocabile.
 4. **Linker/Binder:** Unisce file oggetto e librerie.
 5. **Loader:** Converte indirizzi rilocabili in assoluti e carica in memoria.

9.1 Analisi del Compilatore

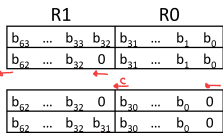
- **Analisi Lessicale:** Rimozione spazi e raggruppamento token.
- **Analisi Sintattica/Semantica:** Controllo struttura e tipi di dati.
- **Ottimizzazione:** Miglioramento del codice per velocità o spazio.
- **Generazione:** Produzione del codice target.

10 Esempi

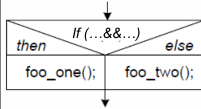
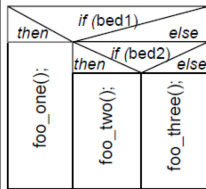
```
1 ;Example: 64Bit Addition
2 LDR r4, =addrOp ;load address of int64 array [op1,op2,res]
3 LDM r4, {r0-r3} ;load op1[r1,r0] and op2[r3,r2]
4 ADDS r0, r0, r2 ;add lower words and set carry
5 ADCS r1, r1, r3 ;add higher words with carry in
6 ADDS r4, #16 ;set r4 to address of res
7 STM r4!, {r0-r1} ;store 64bit result from r0, r1 to res
```

```
1 ; Example: 64Bit Subtraction
2 LDR r4, =addrOp ; load address of int64 array [op1,op2,res]
3 LDM r4, {r0-r3} ; load op1[r1,r0] and op2[r3,r2]
4 SUBS r0, r0, r2 ; sub lower words and set borrow
5 SBCS r1, r1, r3 ; sub higher words with borrow in
6 ADDS r4, #16 ; set r4 to address of res
7 STM r4!, {r0-r1} ; store 64bit result from r0, r1 to res
```

```
1 ;Example for 64bit logical shift left
2 LDR r2, =data64 ;load address of data64
3 LDM r2, {r0-r1} ;load data64 to r0, r1
4 LSLS r1, r1, #1 ;logical shift left r1 by 1
5 MOVS r3, #0 ;r3 = 0
6 LSLS r0, r0, #1 ;lsl r0 by 1 (carry = b31)
7 ADCS r1, r3 ;r1 = r1 + 0 + carry
8 STM r2!, {r0-r1} ;store r0,r1 in data64
```



Equivalente a: (·2)



Thumb-2 Assembly	C Source Code
<pre>LDR R0, =G2 LDR R1, =G1 LDRH R0, [R0] ;R0 <- G2 SXTH R0, R0 LDR R1, [R1] ;R1 <- G1 CMP R0, R1 BGE else1if then: BL foo_one B endif else1if: CMP R0, R1 BLE else2 BL foo_two B endif else: BL foo_three endif:</pre>	<pre>int32_t G1; // signed 32bit int16_t G2; // signed 16bit ... if(G1 > G2) // G2-G1 < 0? { foo_one(); } else if(G1 > G2) // G2-G1 > 0? { foo_two(); } else // G2 == G1 { foo_three(); }</pre>
<pre>LDR R0, =G1 LDR R0, [R0] ;R0 <- G1 MOVS R1, #4 ;R1 = 4 CMN R0, R1 ;G1-(-4) BLE else CMP R0, R1 ;G1-4 BGT else then: BL foo_one B endif else: BL foo_two endif:</pre>	<pre>int32_t G1; // signed 32bit ... if((G1 > -4) && (G1 <= 4)) { foo_one(); } else { foo_thwo(); }</pre>

Thumb-2 Assembly	C Source Code
<pre>0x080000f6 LDR R1, =State 0x080000f8 LDRB R0, [R1] ;R0 <- State 0x080000fa CMP R0, #3 ;Default State R0 - 3 0x080000fc BHI default ;R0 - 3 > 0 0x08000100 LSLS R0, R0, #2 ;R0 = R0 * 4 (offset) * 0x08000102 ADR R1, lookup ;R1 = 0x08000108 0x08000104 LDR R0, [R1, R0] ;R0 = val[lookup + offset] 0x08000106 MOV PC, R0 ** ;PC <- Addr of case lookup: 0x08000108 .word 0x08000130 ;addr of case 0 0x0800010c .word 0x08000136 ;addr of case 1 0x08000110 .word 0x0800013c ;addr of case 2 0x08000114 .word 0x08000142 ;addr of case 3 ... case_0: 0x08000130 BL foo_one 0x08000134 B end_switch case_1: 0x08000136 BL foo_two 0x0800013a B end_switch case_2: 0x0800013c BL foo_three 0x08000140 B end_switch case_3: 0x08000142 BL foo_four 0x08000146 B end_switch default: 0x08000148 MOVS R0, #0 0x0800014a STRB R0, [R1] end_switch: 0x0800014c ...</pre>	<pre>uint8_t State; // unsigned 8bit ... switch(State) { case 0: foo_one(); break; case 1: foo_two(); break; case 2: foo_three(); break; case 3: foo_four(); break; default: State = 0; break; } ...</pre>

1. **Controllo limiti:** Verifica valore di State per saltare subito al default se l’indice è fuori range.
2. **Calcolo offset:** Moltiplicazione del valore per 4 (State << 2) tramite shift logico, poiché ogni puntatore nella look-up table occupa 32 bit (4 byte).
3. **Recupero puntatore:** Lettura dell’indirizzo di destinazione esatto sommando l’offset calcolato all’indirizzo base della tabella.
4. **Branch:** avviare l’esecuzione del caso specifico.

